Rags To Searches - Better Medical Responses Through RAG

Tyler Garfield, Isaac Cota Jr, Atul Triplicane, Ahmad Shamail

May 6, 2026

Abstract

RAG (Retrieval Augmented Generation) is designed to improve LLM accuracy by retrieving relevant information from external sources before generating a response. In this project, we evaluate how well LLMs answer medical (MDCAT style) questions under three settings: a base model, a medically fine tuned version, and the base model augmented with a RAG system using StatPearls (Statpearls Publishing, 2026). We find that the effect of RAG is mixed, with some models improving while others perform worse, largely due to limitations in the knowledge base, which does not cover all questions and can lead to missing or irrelevant context. We also observe that providing large amounts of retrieved text can reduce performance, suggesting that more context is not always better and that summarization or filtering is necessary. Overall, fine tuning leads to more consistent gains by embedding domain knowledge directly, whereas RAG provides more flexible but less reliable improvements that depend heavily on retrieval quality and how the context is presented to the model.

The Problem

The medical field is one of the most data-rich domains, with a vast amount of complex information to understand. LLMs today provide a fast and accessible way to retrieve this information. However, when not given sufficient or relevant context, LLMs are prone to hallucination, which could be catastrophic in medical applications. Therefore, our project aims to evaluate whether a RAG pipeline, which essentially involves feeding retrieved text into an LLM's context, can lead to a more informed and accurate model.

Implementation

Important files in our directory:

```
Rags_To_Searches/
├── corpus
│   └── statpearls
│       └── statpearls_NBK430685
├── data
│   └── 50k.csv
├── eval_local_llm.py
├── creating_whoosh_index.ipynb
├── statpearls.py
└── README.md
```

Code Compilation and Explanation

Compiling The Medical Corpus

The first step in running this project is to compile the medical corpus for use in our RAG system. The corpus was derived from the MedRAG/statpearls HuggingFace repository which, according to README, contains, "9,330 publicly available StatPearl articles through NCBI Bookshelf" (Xiong).

Follow the below steps to ensure your directory remains identical to the one depicted above.

1. Run the following wget command to download the archive:

NOTE: It is paramount that when running the following commands you are in the Rags_To_Searches directory!

```
wget https://ftp.ncbi.nlm.nih.gov/pub/litarch/3d/12/statpearls_NBK430685.tar.gz -P ./
    corpus/statpearls
```

2. Next run the following tar command to unzip the downloaded Stat Pearls file:

```
tar -xzvf ./corpus/statpearls/statpearls_NBK430685.tar.gz -C ./corpus/statpearls
```

3. Next, run the python script that chunks all the medical articles.

```
python src/data/statpearls.py
```

4. Finally, cd to the ./corpus/statpearls/statpearls_NBK430685 directory and run the two following commands.

```
rm *.jpg
rm *.mp4
```

Using The Whoosh Index

File name: *creating_whoosh_index.ipynb*

Since this project aims to show how RAG can be aided through document retrieval, we first start with a pure Lexical Retrieval system using whoosh which was, "built to match strings" ("The Evolution of Information Retrieval: From Lexical to Neural"). While this is a good starting place this limitation of exact matches makes a neural approach much better. However, it must be noted that this project focuses solely on the combination of FAISS and large language models, not FAISS and Whoosh working together.

To ensure the whoosh index creation works the path_to_doc_chuncks variable in the first cell of the notebook must be set to where the corpus chunks are contained!

Figure 1: Where To Change Corp Path

To ensure that the index creation can occur, you MUST have a directory named woosh_index in the same directory as the notebook. The second and third cells in this notebook will then create the whoosh index.

Medical Test Questions Used for Evaluation

The question set that forms the basis of our evaluation model is a 50k long question set from a real medical exam. Note, for purposes of this illustration fitting on this page it only includes the essential table columns.

In the above example question, the correct answer is denoted by the COP column and is Op D or Ileum (Zero Indexing). However, the index evaluation for the purposes of hyperparameter tuning was

Table 1: Medical Question Set

question	Op A	Op B	Op C	Op D	COP
most common site of mucosa...	Stomach	Duodenum	Jejunum	Ileum	3

done with Mean Reciprocal Rank (MRR) within the top 15 relevant documents retrieved. That is:

$$MRR(Q@15) = \frac{1}{|Q|} \sum_{j=1}^{|Q|} \frac{1}{rank_j}$$

Where, $rank_j$ is defined based upon the following criteria. First, apply the Word Net Lemmatizer to the white-space-separated tokens in the retrieved documents and the proper answer, as denoted by the COP column. This lemmatization process "provides meaningful root words that retain their linguistic context"(Singh). Next, remove stop words from the correct answer. Finally, if all tokens in the answer appear somewhere in the document the document is deemed relevant. $rank_j$ is the first relevant documents rank.

For tuning of the BM25 parameter within the range b=0.3-0.9 and k1=0.5-2.0 (Connelly), the best values were found to be $b = 0.3$ and $k1 = 1.2$. Since this project is primarily meant to investigate indexings affects on the RAG pipeline we will go straight to the final MRR value with the optimal hyperparameter for k1 and b. With a final $MRR(Q@15) \approx 0.13$ the takeaway from the attempted Lexical indexing is the following:

First, with respect to the k1 and b parameters, k1 was best when it was a little bit smaller than the default, and b was best when it was significantly smaller than the default. So we know that a low k1 corresponds to no term frequency and a low b corresponds to no length normalization. My hypothesis for diminished term frequency being better is that most of the relevant medical terms only appear once or twice in any given document. Thus the document fequency for a given term carried much more meaningful information that needed more attention then the term frequency. For the low b, since this corpus was created with the specific design of fixed-length chunks for RAG, most chunks were roughly the same size, and size carried very little meaningful information. But some length normalization was still needed, as not all documents were the exact same length.

However, the final conclusion with an MRR at 15 of just about 0.131 is that Whoosh is not best aligned with the use and goal of our RAG project. The lexical nature of the queries significantly hampers our ability to pull a meaningful context. For our purposes, we need our context to be as similar as possible to the given question, but this search needs to be in a semantic space and not a lexical based space. The next thing we will do is devote all our attention to tuning FAISS to give our RAG pipeline the most relevant and precise context, given our query.

Constructing The RAG Pipeline Using FAISS

RAG System Implementation

The Retrieval-Augmented Generation (RAG) pipeline is implemented in `eval_local_llm.py` through the `SemanticRetriever` class. The system operates over a corpus of StatPearls medical documents stored as JSONL chunks. During initialization, all passages are loaded and encoded using the sentence-transformer `all-MiniLM-L6-v2`, which "maps sentences & paragraphs to a 384 dimensional dense vector space" (sentence transformer). The resulting embeddings are normalized and stored in a FAISS `IndexFlatIP` index, with a flat index providing "the most accurate results" (Mathur). Due to normalization, inner product search in this index is equivalent to cosine similarity, enabling efficient semantic retrieval.

At inference time, each MedMCQA question is embedded using the same model. The FAISS index is then queried to retrieve the top-k most semantically similar passages, where the default configuration uses $k = 3$. To ensure compatibility with the model's context window, each retrieved passage is truncated to a fixed length of `MAX_CTX_CHARS = 1200`. These passages are treated as external knowledge and incorporated into the model prompt.

To improve efficiency across repeated runs, the system supports embedding caching via the `--embeddings-cache` flag. This mechanism stores both the FAISS index and the corresponding NumPy embedding matrix, avoiding the need to recompute embeddings for the entire corpus during subsequent evaluations.

Prompting and Inference

The evaluation framework is designed to compare model performance under two conditions: a baseline setting without retrieval and a RAG-enhanced setting with retrieved context. In both cases, the model is instructed to act as a medical expert and respond strictly with a single answer choice.

In the baseline configuration, the model receives only the question and answer options. The system prompt is defined as follows:

> *"You are a medical expert answering multiple-choice questions. Respond with ONLY the letter of the correct answer (A, B, C, or D). Do not add any explanation."*

In the RAG configuration, the retrieved passages are prepended to the question under a designated "Reference passages" section. The prompt is modified to explicitly encourage the use of this external knowledge:

> *"You are a medical expert answering multiple-choice questions. Use the reference passages below to help choose the correct answer. Respond with ONLY the letter of the correct answer (A, B, C, or D). Do not add any explanation."*

Prompt construction is handled dynamically through a helper function that concatenates the top-k retrieved passages with the original question. This ensures consistency in formatting while allowing flexibility in the number and size of retrieved contexts.

Following prompt construction, a HuggingFace causal language model and its corresponding tokenizer are loaded for inference. To improve throughput, the system performs batched inference, where multiple prompts are tokenized and processed simultaneously. The model generates short outputs, typically a single token corresponding to the predicted answer choice.

Since model outputs may occasionally include additional text, a post-processing step extracts the first valid answer letter (A, B, C, or D). Each prediction is then compared against the ground truth label provided in the dataset. The system records the predicted answer, the correct answer, the raw model output, and any retrieved context used during inference.

Results are written incrementally to a CSV file, enabling the evaluation process to resume in case of interruption. Finally, the script computes evaluation metrics including overall accuracy, per-subject performance, and abstention rate. A summary file is also generated in JSON format, containing key statistics such as total questions evaluated, accuracy, and throughput. This structured evaluation framework enables a controlled comparison between standard LLM inference and FAISS-based retrieval-augmented inference, highlighting the impact of semantic retrieval on model performance.

Coding With LLM's

LLMs were used as development and writing assistants throughout the project, but not as a replacement for the actual benchmark results. The measured numbers come from local model runs over MedMCQA; the external LLMs helped with planning, implementation, debugging, and review.

The main AI tools used were Cursor agents with Opus 4.7 and GPT-5.5, along with Gemini and ChatGPT through copy-paste workflows. Cursor agents were especially useful for multi-step tasks because they could inspect project files, ask clarifying questions when requirements were ambiguous, and keep implementation details grounded in the existing code. Gemini and ChatGPT were useful for second-pass sanity checks, code review, and asking whether a design or script made sense.

The most effective prompting strategy was to reduce ambiguity before asking for implementation. Instead of asking broadly for "a RAG project," the prompts defined the research question, comparison groups, data source, vector database, embedding model, and expected planning behavior.

Example planning prompt

```
I am working towards asking a simple question: How well can LLMs perform in medical
    related questions and answers.

My aim is to compare three cases:
1) Mistral 7B -> How well it answers questions raw
2) Mistral 7B + Attached RAG -> How much value does the RAG system add
3) BIOMistral 7B -> how much does fine tuning help

The RAG data we will use is the stats pearl (already chunked in ./corpus)

The vector store i would like to use is FAISS and a simple embedder like all_miniLM-l6
    -v2

Let's plan out the scope and methodology of the project before we move forward, do not
     make any assumptions and ask me in case of any ambiguity
```

This prompt worked well because it gave the assistant the purpose of the project, the experimental arms, and the major technical constraints. It also explicitly asked the assistant not to fill in missing information silently. That reduced the chance of the agent building the wrong thing or making hidden assumptions about the dataset, model, or retrieval setup.

Results

Measuring Performance

For the Whoosh retrieval system, we used Mean Reciprocal Rank at 15 (MRR@15) as our evaluation metric. This measures how high the first relevant document appears in the top 15 results. We chose this metric because our goal was to see if a document containing the correct answer could be retrieved near the top of the rankings or not. A document was considered to be relevant if after lemmatization and removing stop words, it contains all the tokens from the correct answer.

The best performance we achieved was MRR@15 ≈ 0.13, which is considered to be low. This means that even when the relevant documents existed, they were usually not ranked high. This suggests that a lexical only system like Whoosh is not the best method for this project, since it can't capture semantic similarity between the query and documents.

The main RAG evaluation uses 1000 single-answer MedMCQA questions from `data/50k.csv` (we ran the evaluation on UArizona HPC with `Tesla P100-PCIE-16GB GPU`). Each model output is reduced to one answer letter and scored against the ground-truth option.

Condition	Model	Accuracy	Correct / Total	Throughput
Base LLM	Mistral-7B-Instruct-v0.1	45.9%	459 / 1000	0.74 q/s
Base + RAG	Mistral-7B-Instruct-v0.1	51.4%	514 / 1000	0.59 q/s
Medical fine-tune	BioMistral-7B	49.9%	499 / 1000	0.74 q/s

RAG improved the base Mistral model by 5.5 percentage points. BioMistral improved over the same base model by 4.0 percentage points. The important result is not that RAG always beats fine-tuning, but that this simple retrieval system produced a strong improvement without training the model.

The tradeoff is speed. RAG lowered throughput from 0.74 questions per second to 0.59 questions per second because each prompt became longer and included retrieved passages. This is a practical cost: retrieval improves accuracy, but it makes inference slower.

A smaller-model sweep showed that RAG is not automatically helpful:

Model	No RAG	+ RAG	Interpretation
SmolLM2-135M	28.1%	25.0%	Too small to reliably use context
SmolLM2-1.7B	38.2%	41.7%	Large enough for retrieval to help
Llama-3.2-1B	29.2%	14.9%	Non-instruct behavior made context harmful

This suggests that RAG depends on the model's ability to follow instructions and use retrieved text as evidence. A weak or non-instruction-tuned model may treat the extra passages as confusing continuation text rather than reference material.

Error Analysis

The detailed result files make it possible to compare the baseline and RAG runs question by question. Joining `results/bench_base.detail.json` and `results/bench_base_rag.detail.json` by question id gives four useful categories:

Category	Count	Meaning
RAG fixes baseline error	164	Baseline was wrong, RAG was correct
RAG preserves correct answer	350	Both baseline and RAG were correct
RAG fails to help	377	Both baseline and RAG were wrong
RAG hurts	109	Baseline was correct, RAG was wrong

These categories show why the average accuracy improved but also why retrieval is imperfect. RAG helped on many questions, but there were also many cases where the retrieved context was irrelevant, incomplete, or actively distracting.

When RAG Helped

Field	Value
Question	Carbon monoxide effect on O2 dissociation curve is?
Options	A) Shift to right B) Shift to left C) No change D) Linear curve
Ground truth	`B`
Baseline prediction	`A`
RAG prediction	`B`
Context	The dissociation curve also undergoes a leftward shift in carbon monoxide poisoning. CO has a 240-fold greater affinity for hemoglobin than oxygen and will displace oxygen. This favors retention of O2 (keeping hemoglobin in the tense state) on hemoglobin at peripheral tissues. Despite a greater proportion of saturated hemoglobin molecules, total O2 content is decreased because...

In this example, the LLM initially thought the correct answer was A) Shift to right, but with the added context, in particular the first line "The dissociation curve also undergoes a leftward shift in carbon monoxide poisoning..." it corrected itself to B) Shift to left.

When RAG Preserved Correct Answers

Field	Value
Question	Which among the following is the best method to assess intake of fluid in poly trauma patient:
Options	A) Urine output B) CVP C) Pulse rate D) BP
Ground truth	`A`
Baseline prediction	`A`
RAG prediction	`A`
Context	After selecting the appropriate intravenous fluid for replenishment a therapeutic goal must be selected. Several measures can be used to assess volume status and guide therapy. In critically ill patients being taken care of in the intensive care unit, more invasive procedures can be used to monitor more closely the response to therapy, some of these are: Insertion of a urinary catheter to measure urine output Insertion of an arterial line to measure arterial blood pressure and variations in systolic blood pressure...

The retrieved context is effective because it does not merely state "urine output" as an isolated fact, but situates it within a broader clinical framework of fluid assessment in critically ill patients. Specifically,

it highlights that urine output measurement via urinary catheterization is a more direct and reliable method for monitoring a patient's response to fluid resuscitation. While other parameters such as blood pressure and arterial line measurements are mentioned, the context implicitly distinguishes urine output as a primary and sensitive indicator of renal perfusion and overall fluid status.

When RAG Failed To Help

Field	Value
Question	Agoraphobia is -a) Fear of open spacesb) Fear of closed specesc) Fear of heightsd) Fear of crowded places
Options	A) b B) c C) ad D) ab
Ground truth	C
Baseline prediction	D
RAG prediction	D
Context	Agoraphobia: Individuals with this disorder are fearful and anxious in two or more of the following circumstances: using public transportation, being in open spaces, being in enclosed spaces like shops and theaters, standing in line or being in a crowd, or being outside of the home alone. The individual fears and avoids these situations because he/she is concerned that escape may be difficult or help may not be available in the event of panic-like symptoms, or other incapacitating or embarrassing symptoms (e.g., falling or incontinence)...

The retrieved context contains the necessary information, clearly indicating that agoraphobia involves fear of open spaces and situations where escape may be difficult. However, because it also mentions enclosed and crowded spaces as part of the condition, the model is misled and overgeneralizes. Instead of correcting the baseline mistake, the additional details in the context reinforce the incorrect association, leading to the same wrong prediction.

When RAG Hurt

Field	Value
Question	Disadvantage of having a short sprue is?
Options	A) Rapid solidification of metal B) No place for reservoirs C) Incomplete evacuation of gases D) Difficulty in removing casting from investment
Ground truth	C
Baseline prediction	C
RAG prediction	B
Context	Other disadvantages attributable to flapless surgery (when used) such as lack of bone visibility, inability to irrigate the bone, and contraindications in situations requiring alveoloplasty to gain prosthetic space. Inadequate length. Tropical sprue primarily affects both indigenous populations and travelers residing in endemic areas for more than 1 month and is uncommon in visitors staying less ...

In this example, the baseline model correctly identified that a short sprue leads to incomplete evacuation of gases. But the RAG model retrieved irrelevant context about "tropical sprue", which is a medical condition that is completely unrelated to dental casting. This happened because the retriever matched the keyword "sprue" without understanding its specific meaning. It pulled information from the wrong context. This caused the model to be misled by the irrelevant context, and it changed a correct answer to an incorrect one.

Improved Implementation

The initial RAG pipeline highlighted several key limitations, particularly the model's sensitivity to irrelevant or overly verbose retrieved context and the retriever's inability to consistently surface semantically relevant passages. To address these issues, we explored a set of targeted improvements focusing on reasoning augmentation, context compression, and retrieval enhancement.

Chain-of-Thought Reasoning

We first evaluated whether enabling chain-of-thought (CoT) reasoning could improve performance by encouraging the model to reason more explicitly over the provided information. In the baseline setting, CoT produced a modest improvement, increasing accuracy from 45.9% to 46.3%. However, when combined with RAG, performance decreased to 45.1%, suggesting that additional reasoning steps may amplify the impact of noisy or irrelevant retrieved context rather than mitigate it. This indicates that CoT is not inherently beneficial in retrieval-augmented settings where context quality is inconsistent.

Context Summarization

Given earlier observations that large amounts of retrieved text can overwhelm the model, we introduced a summarization step to condense retrieved passages before inference. We evaluated summary lengths of 500 and 750 tokens. Both configurations resulted in identical accuracy (49.1%), which is lower than the original RAG setup. Additionally, summarization significantly reduced throughput, dropping to approximately 0.13 questions per second. These results suggest that while summarization reduces context size, it may also remove critical details necessary for answering medical questions, leading to degraded performance.

Failure Case Example An example of summarization failure highlights an important limitation of this approach. Consider the following retrieved passages:

```
1+ deviation = a slight deviation ...
Mathematically the mean is the sum of the values in a set divided by the number of values ...
Mean Deviation: This is the average of all the values of the total deviation numerical plot ...
```

After summarization, the resulting context was:

```
The mean deviation is a measure of the average deviation of a set of values from the mean.
It is calculated by adding all the values in the set and dividing by the number of values ...
```

Although the summary is factually correct, it introduces ambiguity by emphasizing the word "average" and framing the concept in a purely statistical sense. This causes the model to associate the question with general definitions of mean rather than the specific clinical meaning of mean deviation in ophthalmology. As a result, the model becomes anchored to an incorrect interpretation despite the presence of relevant information in the original passages.

This example illustrates that even high quality summaries can distort task-relevant meaning by oversimplifying domain-specific terminology. It suggests that summarization alone is insufficient and that a stronger, more context-aware model may be required to correctly interpret compressed medical knowledge without losing critical distinctions.

Query Expansion via HyDE

To improve retrieval quality, we implemented Hypothetical Document Embeddings (HyDE), where the model generates a synthetic answer to the query and this generated text is used as the retrieval query instead of the original question. This approach aims to bridge the semantic gap between user queries and relevant documents.

This modification resulted in the most significant improvement across all experiments, increasing accuracy to 55.7%, substantially outperforming both the baseline and standard RAG configurations. Notably, HyDE also improved throughput relative to standard RAG, increasing from 0.18 to 0.24 questions per second. This suggests that better retrieval reduces downstream confusion and allows the model to arrive at answers more efficiently.

Discussion

These results highlight that retrieval quality is the primary bottleneck in RAG systems. Techniques that attempt to improve reasoning or compress context do not address the root issue and may even

degrade performance. In contrast, improving the query representation through HyDE directly enhances the relevance of retrieved documents, leading to more consistent gains.

Overall, this improved implementation demonstrates that semantic alignment between the query and retrieved documents is more critical than increasing model reasoning capacity or reducing context length. Future improvements should therefore focus on retrieval optimization, such as improved embedding models, reranking strategies, or hybrid retrieval approaches.

Conclusion

The conclusion to this project is as follows. RAG is far from perfect. It adds additional time to the query process and it does not always improve a models question answering performance. However, even with no additional model fine tuning the retrieved context can have a notable impact on performance towards the positive direction. If the model is unable to process the extra context however, then it can lead to confusion actually hurting the performance of the model. Thus, when building out a RAG pipeline the choice of model is of great importance as well as ensuring only relevant context is retrieved.

Other possible ways to improve our RAG pipelines performance in the future could be to train the FAISS index to actually better retrieve relevant context. A improvement could also be made in condensing the context being fed into the generating model. Also utilizing FAISS and whoosh together for better retrieval.

References

Connelly, Shane. "Practical BM25 - Part 3: Considerations for Picking B and K1 in Elasticsearch." Elastic Blog, Elastic, 19 Apr. 2018, www.elastic.co/blog/practical-bm25-part-3-considerations-for-picking-b-and-k1-in-elasticsearch.

"The Evolution of Information Retrieval: From Lexical to Neural." iPullRank, 24 Feb. 2026, ipullrank.com/ai-search-manual/ir-evolution.

Mathur, Akash. "Demystifying Faiss, Vector Indexing, and Ann." *Kaggle*, Kaggle, 2024, www.kaggle.com/code/akashmathu faiss-vector-indexing-and-ann.

Sentence Transformers. "all-MiniLM-L6-v2." *Hugging Face*, 2021, huggingface.co/sentence-transformers/all-MiniLM-L6-v2.

Singh, Anoop. "Understanding Wordnet Lemmatizer with NLTK." Medium, 29 Jan. 2025, medium.com/@anoop-singh-dev/understanding-wordnet-lemmatizer-with-nltk-b695458f256a.

"Statpearls." National Center for Biotechnology Information, U.S. National Library of Medicine, 2026, www.ncbi.nlm.nih.gov/books/NBK430685/.

Xiong, Guangzhi, et al. "Benchmarking Retrieval-Augmented Generation for Medicine." arXiv.Org, Cornell University, 23 Feb. 2024, arxiv.org/abs/2402.13178.